

ECEN 662: Estimation & Detection Theory

Lecture 9: Generative Adversarial Network

Tie Liu

Texas A&M University

The power of variational characterizations

- In our previous lecture, we discussed how the idea of *variational characterization* leads to the famous *expectation-maximization (EM)* algorithm

The power of variational characterizations

- In our previous lecture, we discussed how the idea of *variational characterization* leads to the famous *expectation-maximization (EM)* algorithm
- In this supplemental lecture, we shall see, yet again, that the idea of *variational characterization* leads to a highly successful learning algorithm, known as *generative adversarial network (GAN)*, for learning a *generative model*

The power of variational characterizations

- In our previous lecture, we discussed how the idea of *variational characterization* leads to the famous *expectation-maximization (EM)* algorithm
- In this supplemental lecture, we shall see, yet again, that the idea of *variational characterization* leads to a highly successful learning algorithm, known as *generative adversarial network (GAN)*, for learning a *generative model*
- Together, these two examples highlight the power of *variational characterization* across both *inference* and *learning* problems

The power of variational characterizations

- In our previous lecture, we discussed how the idea of *variational characterization* leads to the famous *expectation-maximization (EM)* algorithm
- In this supplemental lecture, we shall see, yet again, that the idea of *variational characterization* leads to a highly successful learning algorithm, known as *generative adversarial network (GAN)*, for learning a *generative model*
- Together, these two examples highlight the power of *variational characterization* across both *inference* and *learning* problems
- These two examples will also help us to understand some of key *differences* between an *inference* problem and a *learning* problem

Learn to generate

- For the problem of *learn-to-generate*, the learner has access to m training data examples $(x_i : i \in [m])$ such that each x_i is drawn i.i.d. from an *unknown* data-generating distribution $p(x)$

Learn to generate

- For the problem of *learn-to-generate*, the learner has access to m training data examples $(x_i : i \in [m])$ such that each x_i is drawn i.i.d. from an *unknown* data-generating distribution $p(x)$
- In addition, the learner also has access to a *random source* Z , which is *easy* to draw samples from

Learn to generate

- For the problem of *learn-to-generate*, the learner has access to m training data examples $(x_i : i \in [m])$ such that each x_i is drawn i.i.d. from an *unknown* data-generating distribution $p(x)$
- In addition, the learner also has access to a *random source* Z , which is *easy* to draw samples from
- The goal is to learn a *generator*, i.e. a conditional distribution $p_\theta(x|z)$, such that:
 - it is *easy* to draw samples of X from $p_\theta(x|z)$ for a given sample of Z
 - the distribution of the generator output $p_\theta(x)$ is very *close* to the unknown data-generating distribution $p(x)$

Learn to generate

- For the problem of *learn-to-generate*, the learner has access to m training data examples $(x_i : i \in [m])$ such that each x_i is drawn i.i.d. from an *unknown* data-generating distribution $p(x)$
- In addition, the learner also has access to a *random source* Z , which is *easy* to draw samples from
- The goal is to learn a *generator*, i.e. a conditional distribution $p_\theta(x|z)$, such that:
 - it is *easy* to draw samples of X from $p_\theta(x|z)$ for a given sample of Z
 - the distribution of the generator output $p_\theta(x)$ is very *close* to the unknown data-generating distribution $p(x)$
- The generator $p_\theta(x|z)$ is commonly known as the *generative model* in the machine learning literature

Generative adversarial network (GAN)

- *Generative adversarial network (GAN)* is a powerful mathematical framework for learning a *generative model*

Generative adversarial network (GAN)

- *Generative adversarial network (GAN)* is a powerful mathematical framework for learning a *generative model*
- There are two distinct features for GAN

Generative adversarial network (GAN)

- *Generative adversarial network (GAN)* is a powerful mathematical framework for learning a *generative model*
- There are two distinct features for GAN
- First, GAN uses a *deterministic* generator $g_{\theta}(z)$ to produce new data samples

Generative adversarial network (GAN)

- *Generative adversarial network (GAN)* is a powerful mathematical framework for learning a *generative model*
- There are two distinct features for GAN
- First, GAN uses a *deterministic* generator $g_\theta(z)$ to produce new data samples
- Second, GAN uses the *Jensen-Shannon (JS) divergence* to measure the distance between the *target* data-generating distribution $p_\theta(x)$ and the *true* data-generating distribution $p(x)$:

$$D_{JS}(p_\theta \| p) = \frac{D_{KL}\left(p_\theta \parallel \frac{p_\theta + p}{2}\right) + D_{KL}\left(p \parallel \frac{p_\theta + p}{2}\right)}{2}$$

Generative adversarial network (GAN)

- *Generative adversarial network (GAN)* is a powerful mathematical framework for learning a *generative model*
- There are two distinct features for GAN
- First, GAN uses a *deterministic* generator $g_\theta(z)$ to produce new data samples
- Second, GAN uses the *Jensen-Shannon (JS) divergence* to measure the distance between the *target* data-generating distribution $p_\theta(x)$ and the *true* data-generating distribution $p(x)$:

$$D_{JS}(p_\theta \| p) = \frac{D_{KL}(p_\theta \| \frac{p_\theta + p}{2}) + D_{KL}(p \| \frac{p_\theta + p}{2})}{2}$$

- Note that unlike the *K-L divergence*, the *J-S divergence* is *symmetric* between its two input distributions

Inference vs. learning

- One may be tempted to use the J-S divergence $D_{JS}(p_\theta \| p)$ as the learning objective, but would quickly come to realization that this learning objective is *not* directly available because the true data-generating distribution $p(x)$ is *unknown*

Inference vs. learning

- One may be tempted to use the J-S divergence $D_{JS}(p_\theta \| p)$ as the learning objective, but would quickly come to realization that this learning objective is *not* directly available because the true data-generating distribution $p(x)$ is *unknown*
- Instead, this learning objective has to be *estimated* from the training data examples $(x_i : i \in [m])$

Inference vs. learning

- One may be tempted to use the J-S divergence $D_{JS}(p_\theta \| p)$ as the learning objective, but would quickly come to realization that this learning objective is *not* directly available because the true data-generating distribution $p(x)$ is *unknown*
- Instead, this learning objective has to be *estimated* from the training data examples $(x_i : i \in [m])$
- This is the key difference between an inference problem such as finding the MLE and a learning problem such as learning a generative model: While for inference problems the challenge is entirely *computational*, learning problems face additional challenge of *estimating* the learning objectives from the training data examples

Inference vs. learning

- One may be tempted to use the J-S divergence $D_{JS}(p_\theta \| p)$ as the learning objective, but would quickly come to realization that this learning objective is *not* directly available because the true data-generating distribution $p(x)$ is *unknown*
- Instead, this learning objective has to be *estimated* from the training data examples $(x_i : i \in [m])$
- This is the key difference between an inference problem such as finding the MLE and a learning problem such as learning a generative model: While for inference problems the challenge is entirely *computational*, learning problems face additional challenge of *estimating* the learning objectives from the training data examples
- It is also important to note that learning algorithms have to take into account that the *estimated* learning objectives are subject to the *sampling noise*, because the number of training data examples is always *finite*

Back to GAN

- The main challenge for GAN is that the J-S divergence $D_{JS}(p_\theta \| p)$ is very difficult to estimate from the training data examples

Back to GAN

- The main challenge for GAN is that the J-S divergence $D_{JS}(p_\theta \| p)$ is very difficult to estimate from the training data examples
- One way to address this challenge is through the following *variational characterization* of the J-S divergence:

$$D_{JS}(p_\theta \| p) = \max_d \frac{\mathbb{E}_{p_\theta}[\log d(X)] + \mathbb{E}_p[\log(1 - d(X))] + 2 \log 2}{2}$$

where the maximization is over a *discriminator* $d : \mathcal{X} \rightarrow [0, 1]$

Back to GAN

- The main challenge for GAN is that the J-S divergence $D_{JS}(p_\theta \| p)$ is very difficult to estimate from the training data examples
- One way to address this challenge is through the following *variational characterization* of the J-S divergence:

$$D_{JS}(p_\theta \| p) = \max_d \frac{\mathbb{E}_{p_\theta}[\log d(X)] + \mathbb{E}_p[\log(1 - d(X))] + 2 \log 2}{2}$$

where the maximization is over a *discriminator* $d : \mathcal{X} \rightarrow [0, 1]$

- An immediate consequence of the above *variational characterization* is that the new objective

$$L(\theta, d) := \mathbb{E}_{p_\theta}[\log d(X)] + \mathbb{E}_p[\log(1 - d(X))]$$

can now be easily estimated from the training data examples:

$$L(\theta, d) \approx \frac{1}{m'} \sum_{i=1}^{m'} \log d(g_\theta(z_i)) + \frac{1}{m} \sum_{i=1}^n \log(1 - d(x_i))$$

where z_i 's are i.i.d. samples of the random source Z

- The problem of learning a generator can thus be posed as a *min-max* optimization problem:

$$\min_{\theta} \max_d \left(\frac{1}{m'} \sum_{i=1}^{m'} \log d(g_{\theta}(z_i)) + \frac{1}{m} \sum_{i=1}^n \log(1 - d(x_i)) \right)$$

- The problem of learning a generator can thus be posed as a *min-max* optimization problem:

$$\min_{\theta} \max_d \left(\frac{1}{m'} \sum_{i=1}^{m'} \log d(g_{\theta}(z_i)) + \frac{1}{m} \sum_{i=1}^n \log(1 - d(x_i)) \right)$$

- Just like the *EM* algorithm, one may consider an *alternating-optimization* strategy for solving the above min-max problem

- The problem of learning a generator can thus be posed as a *min-max* optimization problem:

$$\min_{\theta} \max_d \left(\frac{1}{m'} \sum_{i=1}^{m'} \log d(g_{\theta}(z_i)) + \frac{1}{m} \sum_{i=1}^n \log(1 - d(x_i)) \right)$$

- Just like the *EM* algorithm, one may consider an *alternating-optimization* strategy for solving the above min-max problem
- In practice, one often *parametrizes* the discriminator d as well and optimizes it within the parameter family:

$$\min_{\theta} \max_{\phi} \left(\frac{1}{m'} \sum_{i=1}^{m'} \log d_{\phi}(g_{\theta}(z_i)) + \frac{1}{m} \sum_{i=1}^n \log(1 - d_{\phi}(x_i)) \right)$$

- The problem of learning a generator can thus be posed as a *min-max* optimization problem:

$$\min_{\theta} \max_d \left(\frac{1}{m'} \sum_{i=1}^{m'} \log d(g_{\theta}(z_i)) + \frac{1}{m} \sum_{i=1}^n \log(1 - d(x_i)) \right)$$

- Just like the *EM* algorithm, one may consider an *alternating-optimization* strategy for solving the above min-max problem
- In practice, one often *parametrizes* the discriminator d as well and optimizes it within the parameter family:

$$\min_{\theta} \max_{\phi} \left(\frac{1}{m'} \sum_{i=1}^{m'} \log d_{\phi}(g_{\theta}(z_i)) + \frac{1}{m} \sum_{i=1}^n \log(1 - d_{\phi}(x_i)) \right)$$

- In each iterative step, both *partial* optimization problems can be solved efficiently using *gradient descent (GD)* or *stochastic gradient descent (SGD)* algorithms

Proof of the variational characterization

- First note that

$$\begin{aligned} & \mathbb{E}_{p_\theta}[\log d(X)] + \mathbb{E}_p[\log(1 - d(X))] \\ &= \int_{\mathcal{X}} (p_\theta(x) \log d(x) + p(x) \log(1 - d(x))) dx \end{aligned}$$

Proof of the variational characterization

- First note that

$$\begin{aligned}\mathbb{E}_{p_\theta}[\log d(X)] + \mathbb{E}_p[\log(1 - d(X))] \\ = \int_{\mathcal{X}} (p_\theta(x) \log d(x) + p(x) \log(1 - d(x))) dx\end{aligned}$$

- To maximize the above objective over d , we can choose $d(x) \in [0, 1]$ to maximize

$$p_\theta(x) \log d(x) + p(x) \log(1 - d(x))$$

for *each* $x \in \mathcal{X}$

Proof of the variational characterization

- First note that

$$\begin{aligned}\mathbb{E}_{p_\theta}[\log d(X)] + \mathbb{E}_p[\log(1 - d(X))] \\ = \int_{\mathcal{X}} (p_\theta(x) \log d(x) + p(x) \log(1 - d(x))) dx\end{aligned}$$

- To maximize the above objective over d , we can choose $d(x) \in [0, 1]$ to maximize

$$p_\theta(x) \log d(x) + p(x) \log(1 - d(x))$$

for *each* $x \in \mathcal{X}$

- Consider for each $x \in \mathcal{X}$, the problem of maximizing

$$g(t) := p_\theta(x) \log t + p(x) \log(1 - t)$$

over $t \in [0, 1]$

Proof of the variational characterization

- First note that

$$\begin{aligned}\mathbb{E}_{p_\theta}[\log d(X)] + \mathbb{E}_p[\log(1 - d(X))] \\ = \int_{\mathcal{X}} (p_\theta(x) \log d(x) + p(x) \log(1 - d(x))) dx\end{aligned}$$

- To maximize the above objective over d , we can choose $d(x) \in [0, 1]$ to maximize

$$p_\theta(x) \log d(x) + p(x) \log(1 - d(x))$$

for *each* $x \in \mathcal{X}$

- Consider for each $x \in \mathcal{X}$, the problem of maximizing

$$g(t) := p_\theta(x) \log t + p(x) \log(1 - t)$$

over $t \in [0, 1]$

- Setting $g'(t) = 0$ gives:

$$\frac{p_\theta(x)}{t} - \frac{p(x)}{1 - t} = 0 \quad \Rightarrow \quad t = \frac{p_\theta(x)}{p_\theta(x) + p(x)}$$

- We thus have

$$\begin{aligned}
& \max_{d(x) \in [0,1]} (p_\theta(x) \log d(x) + p(x) \log(1 - d(x))) \\
&= p_\theta(x) \log \frac{p_\theta(x)}{p_\theta(x) + p(x)} + p(x) \log \left(1 - \frac{p_\theta(x)}{p_\theta(x) + p(x)} \right) \\
&= p_\theta(x) \log \frac{p_\theta(x)}{p_\theta(x) + p(x)} + p(x) \log \frac{p(x)}{p_\theta(x) + p(x)}
\end{aligned}$$

and hence

$$\begin{aligned}
& \max_d (\mathbb{E}_{p_\theta}[\log d(X)] + \mathbb{E}_p[\log(1 - d(X))]) \\
&= \int_{\mathcal{X}} \left(p_\theta(x) \log \frac{p_\theta(x)}{p_\theta(x) + p(x)} + p(x) \log \frac{p(x)}{p_\theta(x) + p(x)} \right) dx \\
&= D_{KL} \left(p_\theta \left\| \frac{p_\theta + p}{2} \right\| \right) + D_{KL} \left(p \left\| \frac{p_\theta + p}{2} \right\| \right) - 2 \log 2
\end{aligned}$$